Recce: An End-to-End Automation Platform for AI-Assisted Film Location Scouting

Arun Sharma

June 2026

Abstract

Recce is a deployable film-location scouting system that converts screenplay material into structured location briefs, visually ranked real-world candidates, route plans, mood-board references, and shoot-day logistics packets. The system combines language-model scene understanding, server-side map and imagery services, deterministic fallback modules, open location and permit data, route optimization, sun-window computation, weather summaries, and a production-facing React workspace. This white paper documents the current architecture, execution model, test strategy, and Kubernetes deployment path. It is written as both a technical reference and a starter guide for running, testing, and scaling Recce.

1 Executive Summary

Film location scouting is a high-context workflow. A location manager must read scene intent, infer production constraints, search for real places, judge visual fit, understand permit and access requirements, plan a scout route, and produce usable notes for the production team. Recce compresses that workflow into a single tool surface. The application accepts a scene or full script, extracts one or more location briefs, produces candidate locations, scores those candidates against the director's brief, supports shortlist selection on a map, optimizes a scout day, and generates a logistics packet.

Core contribution: Recce moves beyond keyword search. It treats each candidate as an operational scouting decision: does the real place visually fit the scene, can it be reached efficiently, what permits or restrictions are implied, what sun and weather conditions matter, and what starter shot list supports the scene?

Current implementation: The repository contains a FastAPI backend and a Vite/React frontend served by a single production container. Demo mode works without API keys through bundled briefs and candidates, deterministic script segmentation, demo weather, and placeholder concept frames. Live mode can be enabled through Gemini, Google Maps, OSRM, weather providers, and normalized permit datasets.

2 Application Screenshot

Figure 1 shows the current production-facing workspace: script input and filters on the left, map in the center, and the generated shoot-day packet on the right.

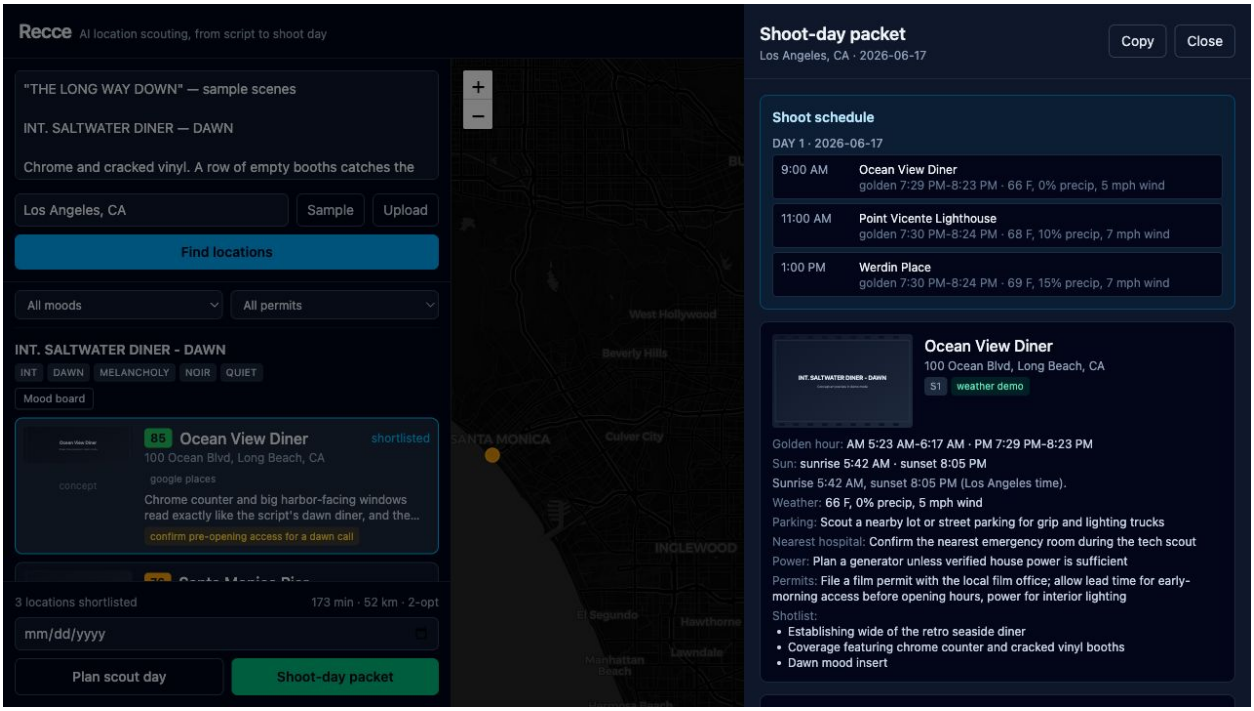

Figure 1: Recce demo workflow after loading the bundled sample, finding candidates, planning a route, and opening the shoot-day packet.

3 System Goals

Production usefulness: The output must be usable by location managers, line producers, and directors. That means the system cannot stop at a ranked search list. It must expose constraints such as access, permit risk, weather, sun windows, driving time, and crew logistics.

Demo reliability: The full workflow must run without API keys. This is important for review, education, hackathon demos, and early design-partner evaluation. Demo mode therefore uses bundled data, deterministic fallbacks, and placeholder images.

Live-mode extensibility: The same code path must support real service providers when keys and URLs are configured. Candidate search, image scoring, routing, weather, and mood-board generation are encapsulated so each live provider can be switched on independently.

Scalable deployment: The production artifact is a single HTTP service. Kubernetes deployment adds replicas, health probes, autoscaling, disruption protection, network policy, and non-root execution.

4 End-to-End Workflow

Scene analysis: The backend receives raw screenplay text through `/api/analyze`. In live mode, Gemini extracts structured `SceneBrief` objects. In demo or fallback mode, `script_analysis.py`

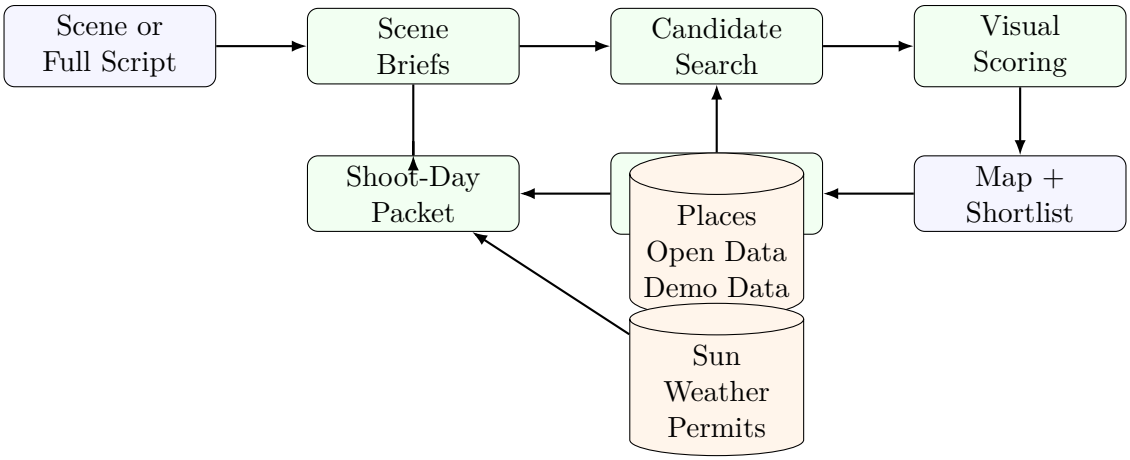

Figure 2: High-level Recce workflow from screenplay input to shoot-day packet.

segments screenplay sluglines and derives location type, time of day, character cues, mood hints, visual keywords, practical needs, and search queries.

Candidate generation: `places.py` calls Google Places when a Maps key is configured. Without a key, it serves bundled candidates. `permits.py` enriches candidates with normalized open-data location records, permit flags, permit contacts, and restrictions. This allows the UI to filter by permit risk even in demo mode.

Visual scoring: When Gemini and Maps are both configured, the system fetches the candidate's best available image, preferring venue photography over Street View, and asks Gemini to score the image against the director's brief. In demo mode, candidate scores and rationales come from bundled data plus open-data priors.

Route planning: `routing.py` uses OSRM Trip routing when `RECCE_OSRM_URL` is configured. If OSRM is unavailable, the system falls back to a deterministic nearest-neighbor seed followed by 2-opt improvement over haversine distances.

Packet generation: `packet.py` combines shortlisted candidates, briefs, sun windows from `astral`, weather summaries, permit metadata, concept-frame URLs, and Gemini or canned production notes. The packet includes a multi-day schedule that rolls over after four locations per day.

5 Backend Architecture

Module	Responsibility
`routes.py`	FastAPI route surface for health, demo scene, analysis, segmentation, candidates, route, packet, image proxying, and moodboard generation.
`schemas.py`	Pydantic models for briefs, scenes, candidates, route results, packets, weather summaries, schedule stops, and request bodies.

Module	Responsibility
`gemini.py`	Live Gemini integration for scene extraction, visual scoring, location notes, and image generation provider calls.
`script_analysis.py`	Deterministic screenplay segmentation and fallback brief generation.
`places.py`	Google Places, Street View, Place Photos, geocoding, demo candidates, and safe SVG placeholders.
`permits.py`	Open-data location records, permit enrichment, source metadata, restrictions, and candidate merge logic.
`moodboard.py`	Prompt construction, live image-byte generation, packet-safe data URLs, and placeholder concept-frame URLs.
`routing.py`	Route result construction, OSRM handoff, haversine distance, nearest-neighbor seed, and 2-opt fallback.
`osrm.py`	Optional OSRM Table and Trip HTTP client.
`astro.py`	Time-zone approximation and golden-hour, sunrise, and sunset calculations.
`weather.py`	Demo, Open-Meteo, and OpenWeather forecast summaries.
`packet.py`	Final packet assembly, permit-note merging, weather injection, concept image URLs, and schedule construction.

6 Frontend Architecture

The frontend is a Vite/React/TypeScript application with a dense scouting workspace. It intentionally avoids a marketing-style landing page because the target user needs a working operational tool. The first viewport contains script input, base city, upload controls, scene filters, candidate cards, map state, route controls, and packet generation.

Core views:

- **Script intake:** Paste screenplay text, load the sample, or upload a text-like script file.
- **Scene selection:** Full scripts can be segmented into scene checkboxes before candidate search.
- **Candidate review:** Candidate cards show real imagery, concept placeholders, match scores, source labels, permit badges, rationales, and flags.
- **Map view:** Leaflet renders selected candidates and route context with keyless OpenStreetMap tiles.
- **Packet drawer:** The generated packet shows schedule, concept frames, sun windows, weather, parking, power, permit notes, restrictions, and starter shot lists.

7 API Surface

Endpoint	Method	Purpose
`/api/health`	GET	Returns status, demo mode, key availability booleans, OSRM availability, model name, and weather provider.
`/api/demo/scene`	GET	Returns the bundled sample screenplay.
`/api/script/segments`	POST	Segments a full script into deterministic scene records.
`/api/analyze`	POST	Produces structured scene briefs from screenplay text.
`/api/candidates`	POST	Produces candidate locations for one or more briefs.
`/api/route`	POST	Optimizes a scout-day route over shortlisted candidates.
`/api/packet`	POST	Builds the shoot-day logistics packet.
`/api/streetview`	GET	Proxies Street View or returns a safe placeholder image.
`/api/placephoto`	GET	Proxies a Places photo or returns a safe placeholder image.
`/api/moodboard`	POST	Generates or placeholders a scene concept frame.

8 Data Contracts

The core models are intentionally small and typed. A `SceneBrief` contains the scouting requirements inferred from the script. A `Candidate` contains a real or demo location, visual score, imagery URL, permit flags, and source metadata. A `RouteResult` contains ordered `RouteStop` objects plus distance, duration, and route method. A `Packet` contains locations, schedule days, production title, date, base city, notes, weather, sun windows, permits, and shot lists.

Backward compatibility: New model fields use defaults. Existing demo JSON and older clients continue to parse because added fields are optional or defaulted in Pydantic.

9 Testing Strategy

The repository implements eight practical test categories. All backend tests run in demo mode by default through `backend/tests/conftest.py`. This prevents local API keys from changing CI behavior.

Type	Coverage	Command
Unit	Script parsing, permit enrichment, routing, weather, sun math, mood prompts, image placeholders.	`make test-unit`
Integration	API route interactions, response parsing, mocked OSRM and Open-Meteo behavior.	`make test-integration`
End-to-end	Full script-to-packet workflow through the API.	`make test-e2e`
Smoke	Fast health and minimum viable workflow checks.	`make test-smoke`
Contract	OpenAPI endpoints, Pydantic response models, Kubernetes object contracts.	`make test-contract`

Type	Coverage	Command
Regression	Demo-scene stability and packet schedule rollover behavior.	`make test-regression`
Performance	Local demo pipeline and route optimizer runtime budgets.	`make test-performance`
Security/config	Secret hygiene, placeholder escaping, non-root Kubernetes settings, network policy.	`make test-security`

Smoke runner: A no-dependency Python script can validate a running service locally, in Kubernetes, or on Cloud Run:

```
python3 scripts/smoke_api.py --base-url http://127.0.0.1:8000
python3 scripts/smoke_api.py --base-url https://recce-216756172879.us-central1.run.app
```

Frontend checks: The frontend target runs TypeScript build, a static UI/API surface smoke check, and ESLint:

```
make test-frontend
```

10 Kubernetes Deployment Architecture

The Kubernetes base under `k8s/base` uses Kustomize and includes a namespace, config map, optional secret reference, deployment, service, horizontal pod autoscaler, pod disruption budget, network policy, and ingress.

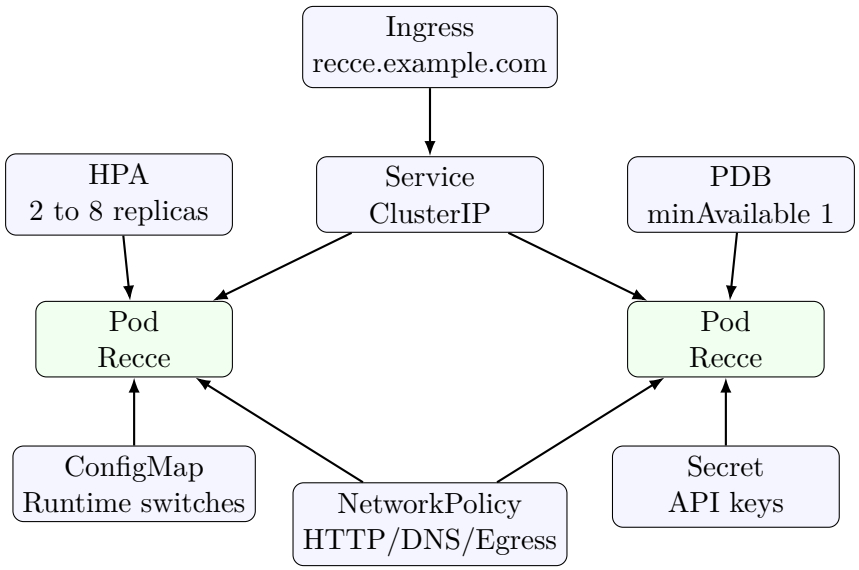

Figure 3: Kubernetes deployment topology for Recce.

Security posture: The container runs as UID `10001`, disallows privilege escalation, drops Linux capabilities, uses `RuntimeDefault` seccomp, uses a read-only root filesystem, and mounts an `emptyDir` only at `/tmp`. Secrets are not committed. The example secret file contains empty values only.

Scalability posture: The deployment starts with two replicas, uses rolling updates with `maxUnavailable: 0`, exposes readiness, liveness, and startup probes on `/api/health`, and autos-scales between two and eight replicas based on CPU and memory utilization.

11 Starter Guide

11.1 Local Demo Mode

Demo mode requires no API keys.

```
git clone https://github.com/arunshar/recce.git
cd recce
make install
make backend    # terminal 1, http://localhost:8000
make frontend   # terminal 2, http://localhost:5173
```

Open http://localhost:5173, click **Sample**, then click **Find locations**. Select candidates, plan the scout day, and generate the shoot-day packet.

11.2 Single-Port Production Build

```
make build
cd backend
. .venv/bin/activate
GEMINI_API_KEY= GOOGLE_MAPS_API_KEY= uvicorn app.main:app --port 8000
```

This serves both the API and built frontend from http://localhost:8000.

11.3 Docker

```
docker compose up --build
python3 scripts/smoke_api.py --base-url http://127.0.0.1:8000
```

11.4 Live Mode

Set the keys and optional providers through environment variables:

```
GEMINI_API_KEY=... \
GOOGLE_MAPS_API_KEY=... \
RECCE_WEATHER_PROVIDER=open-meteo \
docker compose up --build
```

Optional switches:

- `RECCE_OSRM_URL`: OSRM service base URL.

- `RECCE_LOCATION_DATA`: path to normalized open location JSON.
- `RECCE_WEATHER_PROVIDER`: `demo`, `open-meteo`, or `openweather`.
- `OPENWEATHER_API_KEY`: required only for OpenWeather.

11.5 Open-Data Ingestion

The ingestion script can normalize NYC permits, San Francisco filming locations, or a CSV into the backend data schema:

```
python3 scripts/data_ingest/ingest_open_locations.py nyc-permits \
  --limit 5000 \
  --out backend/app/data/locations.json

python3 scripts/data_ingest/ingest_open_locations.py sf-filming \
  --out backend/app/data/locations.json
```

11.6 Kubernetes

Create secrets only if live providers are needed:

```
kubectl create namespace recce
kubectl create secret generic recce-secrets \
  -n recce \
  --from-literal=GEMINI_API_KEY='...' \
  --from-literal=GOOGLE_MAPS_API_KEY='...' \
  --from-literal=OPENWEATHER_API_KEY='...'
```

Deploy and smoke-test:

```
kubectl apply -k k8s/base
kubectl rollout status deployment/recce -n recce
scripts/k8s_smoke.sh
```

12 Operational Best Practices

Use stable image tags: The base manifests default to `arunsharma08/recce:latest` for convenience. Production rollouts should pin immutable image tags or digests.

Keep demo and live modes separate: Demo mode is useful for no-key review and CI. Live mode should run in a namespace with explicit secrets, provider quotas, and observability.

Protect provider keys: Keys stay server-side. Do not expose Google or Gemini keys to the browser. Use Kubernetes Secrets or an external secret manager.

Track provider fallbacks: OSRM, weather, image generation, and Gemini scoring all have fallbacks. This keeps demos reliable, but production monitoring should record whether a request used live services or fallback behavior.

Run smoke tests after deploy: The API smoke script exercises the real service path from health to packet. It should run after every deployment and before demoing a new public URL.

13 Current Verification Status

The current repository includes backend, frontend, and Kubernetes-manifest test coverage. The local verification suite includes:

```
make test
make test-frontend
python3 scripts/smoke_api.py --base-url http://127.0.0.1:8000
kubectl kustomize k8s/base
```

If Docker is running, the next production-grade checks are:

```
docker compose up --build
python3 scripts/smoke_api.py --base-url http://127.0.0.1:8000
```

If a Kubernetes cluster is available:

```
scripts/k8s_smoke.sh
```

14 Limitations and Future Work

PDF and Fountain parsing: The current browser upload path reads text-like scripts. Rich PDF parsing and Fountain-specific syntax support can be added as a separate parser layer.

Formal load testing: The performance tests are deterministic local guardrails. A production system should add a separate load profile, for example k6 or Locust, against live and demo modes.

Provider observability: Future deployments should expose structured metrics for provider call success, fallback usage, route method, packet latency, and candidate scoring latency.

Permit data completeness: The current permit layer supports normalized open-data records and a seed demo corpus. A market-ready version should ingest jurisdiction-specific schemas, normalize recency, and separate historical filming records from active permit requirements.

15 Conclusion

Recce is now structured as a full-stack, testable, deployable scouting platform. It supports deterministic demos, live service integration, typed contracts, frontend workflow coverage, API smoke testing, and Kubernetes deployment practices. The core engineering pattern is modular fallback: every expensive or external capability has a local deterministic path, while production configuration can progressively enable live imagery, vision scoring, routing, weather, and permit data.